

ICT703 Programming

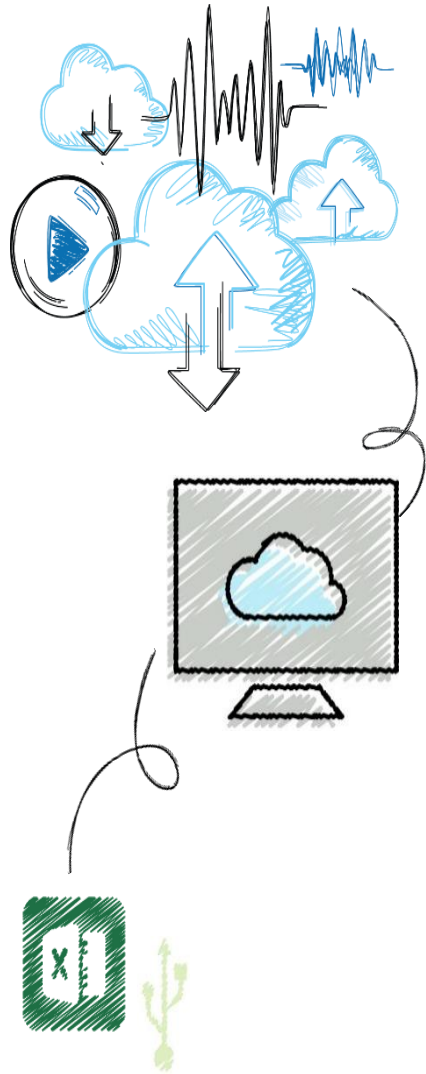
Module 7

Advanced Functions



Objectives

1. Unknown number of arguments
2. Parameters with a default value
3. Keyword arguments
4. Returning multiple values
5. The main function
6. A recursive function





Unknown Number of Arguments

- Put * before the parameter means we can pass many values into the function
 - We have used the print function with different number of arguments
 - Inside the function refer to the parameters like a list

```
def greet(*names):
```

```
    print ('Hello ', names[0], ', ', names[1], ', ', names[3])
```

```
greet('Steve', 'Bill', 'Yash')
```

Hello Steve, Bill, and Yash



Parameter with Default Value

- While defining a function, its parameters may be assigned default values.

```
def greet(name = 'Guest'):
    print ('Hello', name)
```

```
greet()
greet('Steve')
```

```
Hello Guest
Hello Steve
```



Function with Keyword Arguments

- A function can be called by passing parameter values using the parameter names in any order.

```
def greet(firstname, lastname):  
    print ('Hello', firstname, lastname)
```

```
greet(lastname='Jobs', firstname='Steve') # passing parameters in any order
```

Hello Steve Jobs



Returning Multiple Values

- While defining a function, we can return multiple values as a tuple

```
def lookupPerson(name) :  
    age = lookupAge(name)  
    weight = lookupWeight(name)  
    return (age, weight)  
  
print(lookupPerson("Andrew"))  
( 'young', 'skinny' )  
  
(age, weight) = lookupPerson("Andrew")  
print(age)  
young
```



Defining a Main Function (1 of 2)

- **main** serves as the entry point for the program
 - Usually expects no arguments and returns no value
- Definition of **main** and other functions can appear in no particular order in the script
 - As long as **main** is called at the end of the script



Defining a Main Function (2 of 2)

```
"""
```

File: computesquare.py

Illustrates the definition of a main function. """

```
def main():
```

```
    """ The main function for this script. """
```

```
    number = float(input("Enter a number: "))
```

```
    result = square(number)
```

```
    print("The square of", number, "is", result)
```

```
def square(x):
```

```
    """Returns the square of x."""
```

```
    return x * x
```

```
# The entry point for program execution
```

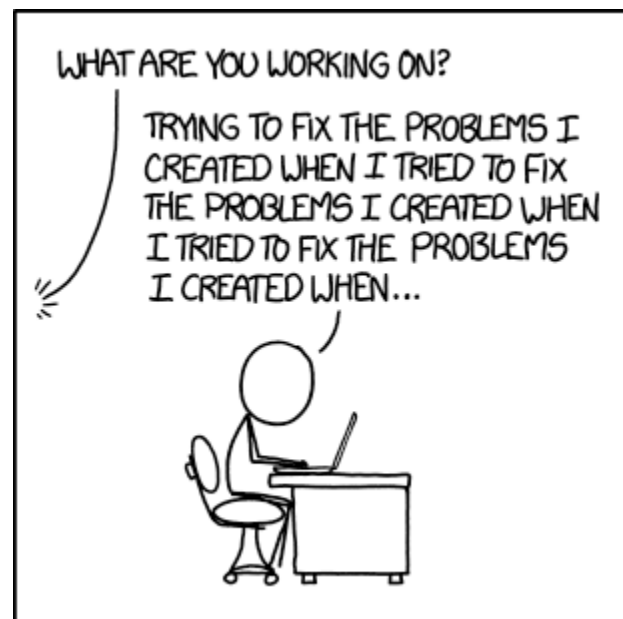
```
if __name__ == "__main__":
```

```
    main()
```



Design with Recursive Functions

- In top-down design, you decompose a complex problem into a set of simpler problems and solve these with different functions
- In some cases, you can decompose a complex problem into smaller problems of the same form
 - Subproblems can be solved using the same function
 - This design strategy is called **recursive design**
 - Resulting functions are called **recursive functions**





Defining a Recursive Function (1 of 2)

- A recursive function is a function that calls itself
 - To prevent function from repeating itself indefinitely, it must contain at least one selection statement
 - Statement examines **base case** to determine whether to stop or to continue with another **recursive step**

- To convert **displayRange** to a recursive function:

```
def displayRange(lower, upper):
```

```
    """Outputs the numbers from lower through upper."""
```

```
    while lower <= upper:
```

```
        print(lower)
```

```
        lower = lower + 1
```

- You can replace loop with a selection statement and assignment statement with a **recursive call**
-



Defining a Recursive Function (2 of 2)

- Making **displayRange** recursive (continued):

```
def displayRange(lower, upper):
```

```
    """Outputs the numbers from lower through upper."""
```

```
    if lower <= upper:
```

```
        print(lower)
```

```
        displayRange(lower + 1, upper)
```

- Most recursive functions expect at least one argument



Defining a Recursive Function (2 of 2)

Assume that remaining is a positive integer

```
def hi_recursive(remaining):
```

The base case

```
    if remaining == 0:
```

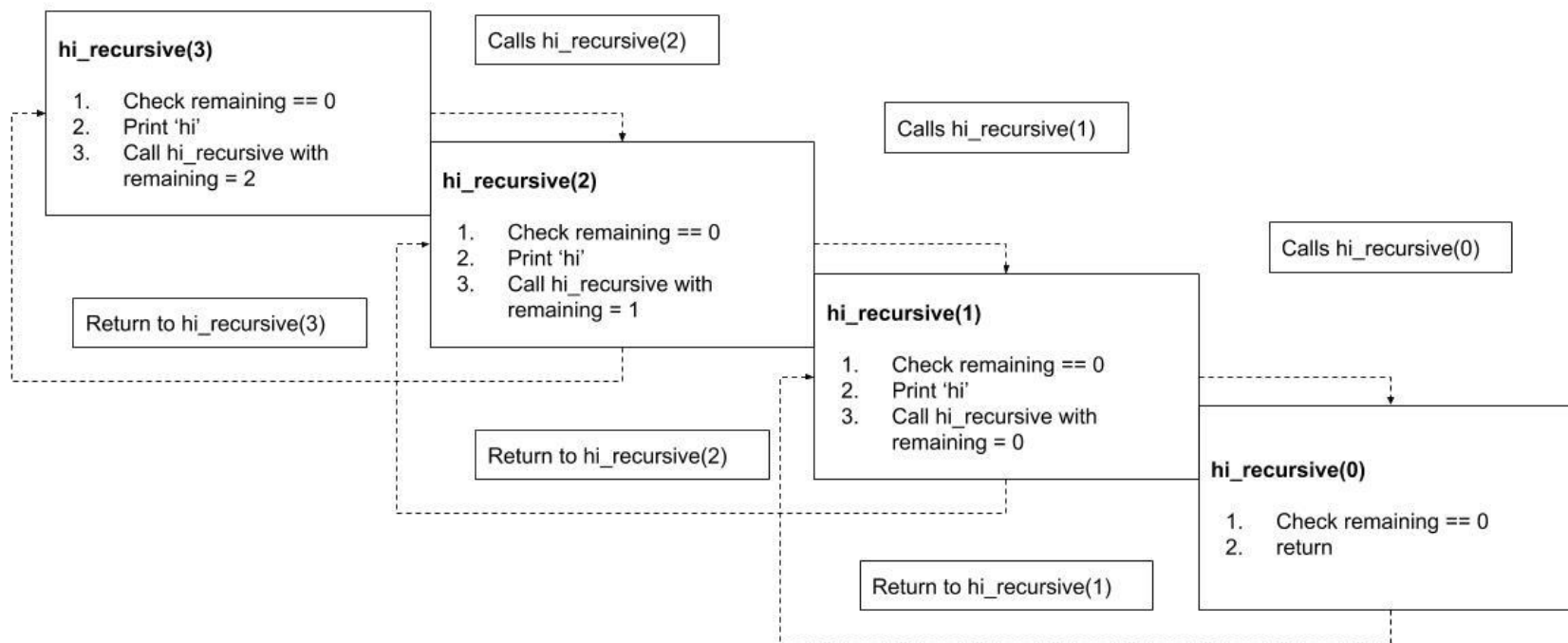
```
        return
```

```
    print('hi')
```

Call to function, with a reduced remaining count

```
    hi_recursive(remaining - 1)
```

- The base case for us is if the remaining variable is equal to 0 i.e. how many remaining "hi" strings we must print. The function simply returns.
- After the print statement, we call hi_recursive again but with a reduced remaining value. This is important!
- If we do not decrease the value of remaining the function will run indefinitely. Generally, when a recursive function calls itself the parameters are changed to be closer to the base case.
- Let's visualize how it works when we call hi_recursive(3):



After the function prints 'hi', it calls itself with a lower value for remaining until it reaches 0.

At zero, the function returns to where it was called in `hi_recursive(1)`, which returns to where it was called in `hi_recursive(2)` and that ultimately returns to where it was called in `hi_recursive(3)`.



Why not use a Loop?

- All traversal can be handled with loops. Even so, some problems are often easier solved with recursion rather than iteration.
 - A common use case for recursion is tree traversal
 - Traversing through nodes and leaves of a tree is usually easier to think about when using recursion.
 - Even though loops and recursion both traverse the tree, they have different purposes
 - loops are meant to repeat a task
 - whereas recursion is meant to break down a large task into smaller tasks.
 - Recursion allows us to break a large task down to smaller tasks by repeatedly calling itself. A recursive function requires a base case to stop execution, and the call to itself which gradually leads to the function to the base case. It's commonly used in trees, but other functions can be written with recursion to provide elegant solutions
-